

VERSIONS- KONTROLLE



DHBW
Duale Hochschule
Baden-Württemberg

MOTIVATION

!?"Oh nein! Ich hätte gerne meine Änderungen von vor 2h wieder!"

!?"Mist! Ich habe die Datei gelöscht!"

!?"Lebenslauf_v5_neu_final_FINAL2.docx

Versionskontrolle bedeutet das Protokollieren von Änderungen an einer Datei oder einer Anzahl von Dateien über die Zeit hinweg, so dass Änderungen nachvollzogen und zu jedem Zeitpunkt auf ältere Versionen zugegriffen werden kann.

ANWENDUNG

💡 Softwareentwicklung

💡 Dokumentation

💡 Texte

💡 Grafikdesign

BEISPIELE

- Office 365
- Google Docs
- Dropbox

WAS IST VERSIONSKONTROLLE?

GRUNDIDEEN 💡

- Angabe der Version oder eines Zeitstempels im Dateinamen
- Angabe innerhalb einer Datei in den sog. Metadaten
- Anlegen eines Änderungsprotokolls
- Versionierung z.B. durch Cloud-Storage Systeme (Google Drive, Dropbox,...)
- Oder bei Software: Version Control Systems (VCS)

VERSIONSKONTROLLE IN DER SOFTWAREENTWICKLUNG

! HERAUSFORDERUNGEN

- ! Viele Dateien, die sich stetig ändern
- ! Unterschiedliche Versionen der Software (z.B. Release, Beta,...)
- ! Unterschiedliche Entwicklungszweige (z.B. Feature-Entwicklung, Bugfixing)
- ! Personen überschreiben gegenseitig ihre Änderungen
- ! Viele Entwickler:innen, die gleichzeitig an den Dateien arbeiten
- ! Wer hat welche Änderungen vorgenommen?
- ! Mein Kollege braucht die Änderungen, die ich auf meiner Maschine vorgenommen habe

VERSIONSKONTROLLE IN DER SOFTWAREENTWICKLUNG

ZIELE

-  Ermöglichen von kollaborativen Arbeiten und paralleler Entwicklung
-  Nachvollziehbarkeit von Änderungen
-  Protokollieren aller Änderungen
-  Vermeidung von gegenseitigen Überschreibungen
-  Darstellung von Unterschieden in verschiedenen Versionen

FUNKTIONSWEISE VON VCS

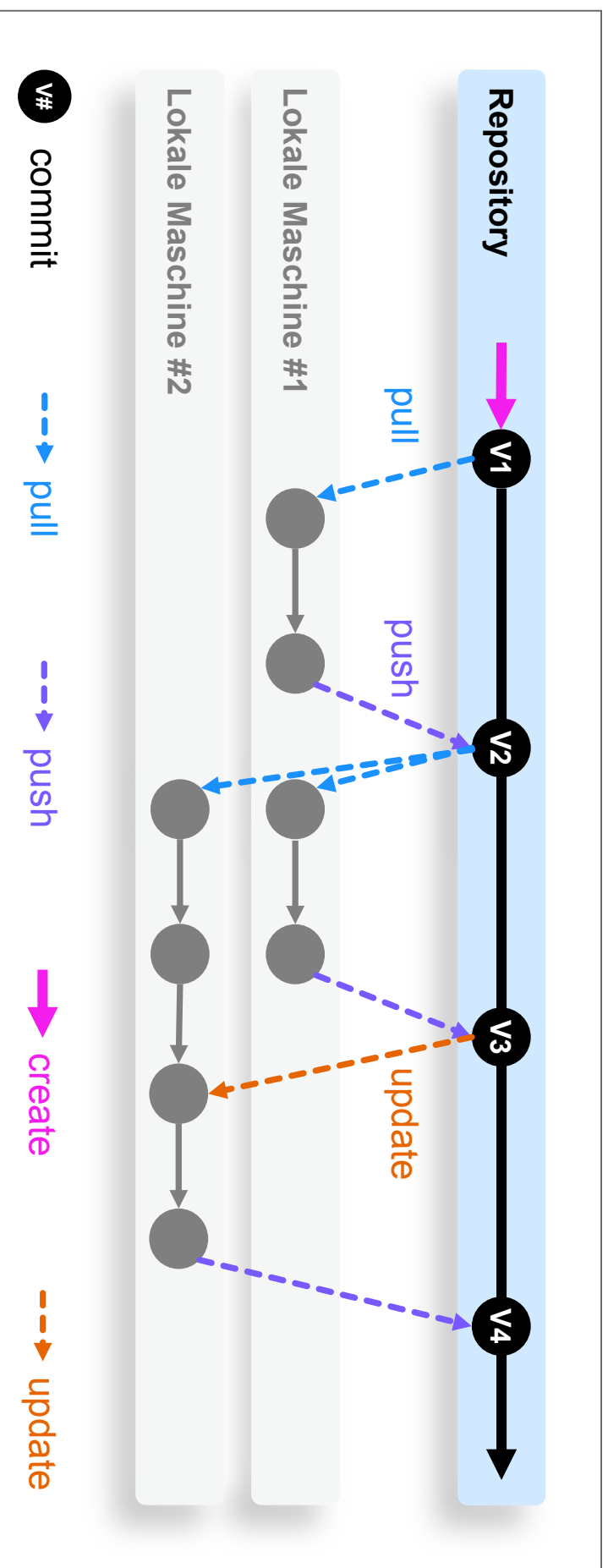
VCS = Version Control System 💡

- create: Erstellen einer Ablage (sog. Repository)
- add: Hinzufügen von Dateien zum Repository
- commit: Speichern von Änderungen im Repository;

Jede Änderung bzw. Jeder Commit erzeugt eine neue Version

- push: Übertragen der Änderungen auf einen zentralen Server (optional)
- pull: Herunterladen der Änderungen von einem zentralen Server
- update: Entwickler aktualisieren ihre lokale Arbeitskopie auf die neueste Version
- revert: Entwickler können ihre Arbeitskopie auf frühere Versionen zurücksetzen

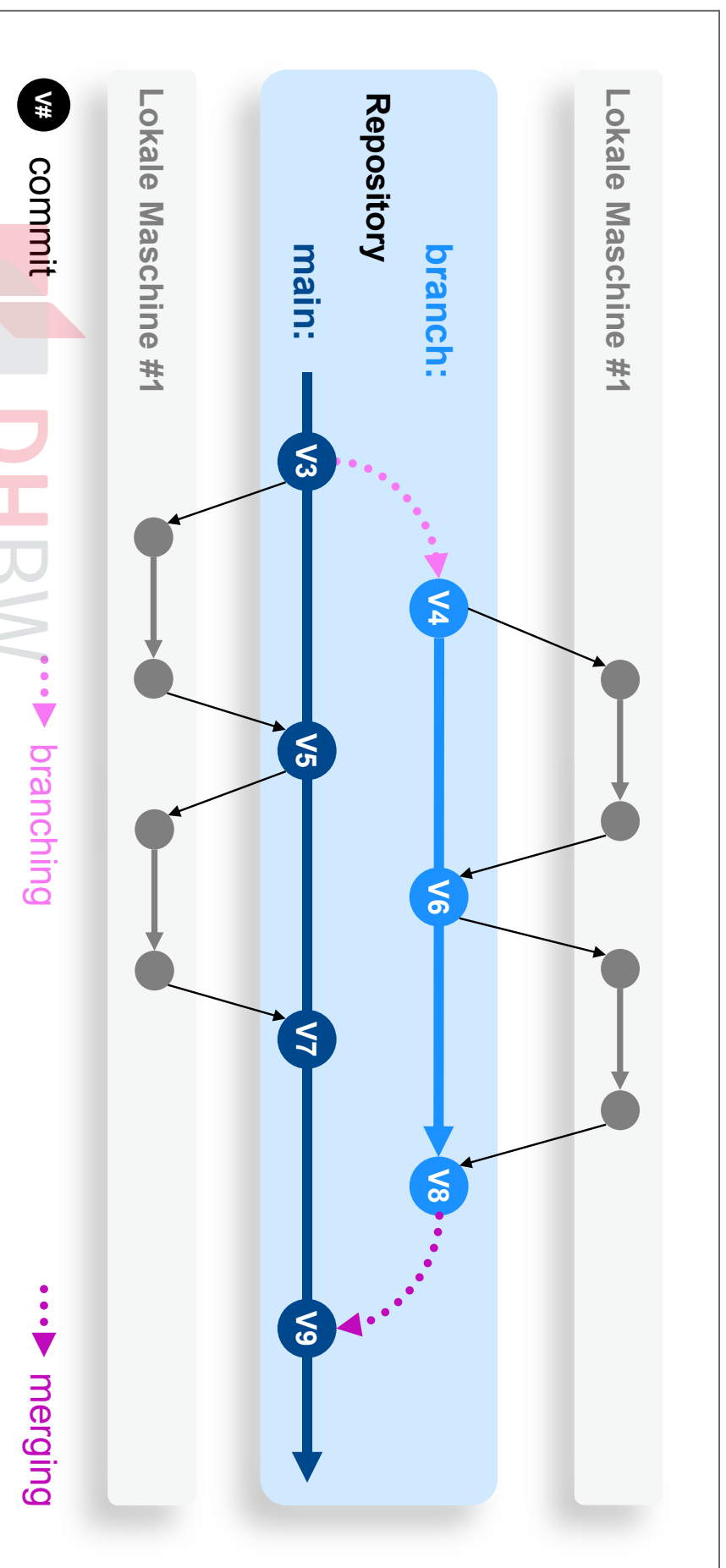
FUNKTIONSWEISE VON VCS



FUNKTIONSWEISE VON VCS

- Oftmals paralleles Entwickeln an unterschiedlichen „Baustellen“ eines Projektes
- branching: VCS ermöglichen dazu das Abspalten von Entwicklungszweigen
Es wird eine Kopie der aktuellen Version erstellt, auf der weiterentwickelt wird
- branch: Entwicklung von Teilprojekten parallel in eigenen Versionspfaden
- merge: Zusammenführen von Änderungen aus verschiedenen Entwicklungszweigen

FUNKTIONSWEISE VON VCS



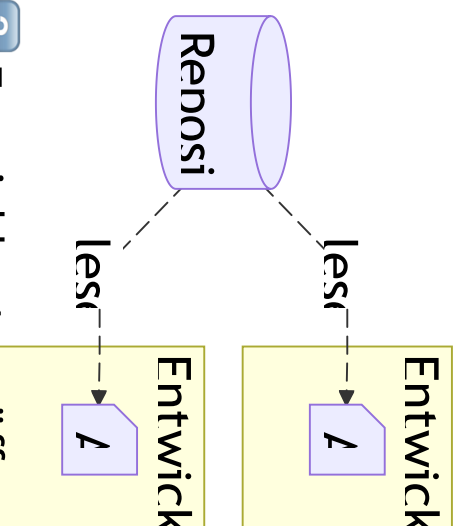
GEMEINSAMES EDITIEREN

HERAUSFORDERUNGEN

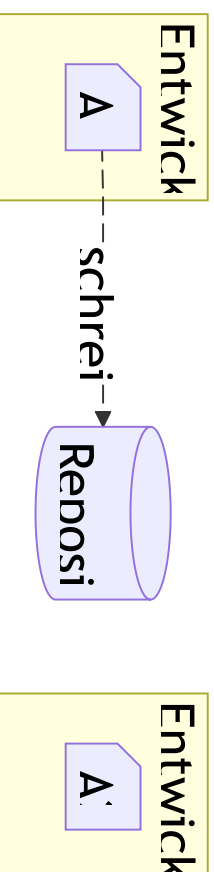


DHBW
Duale Hochschule
Baden-Württemberg

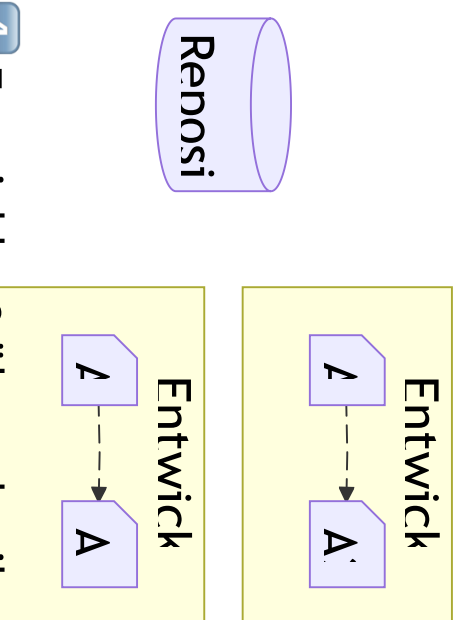
1 2 Entwickler lesen die gleiche Datei



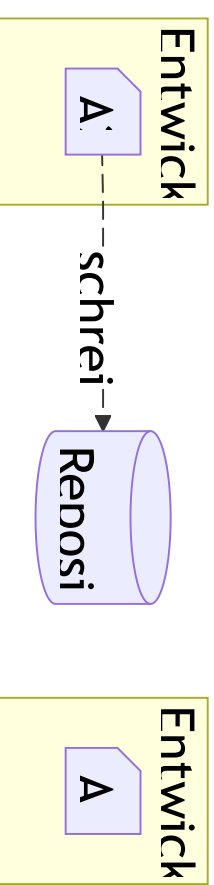
3 Entwickler 1 veröffentlicht seine Änderungen zuerst



2 Beide bearbeiten ihre jeweilige Kopie



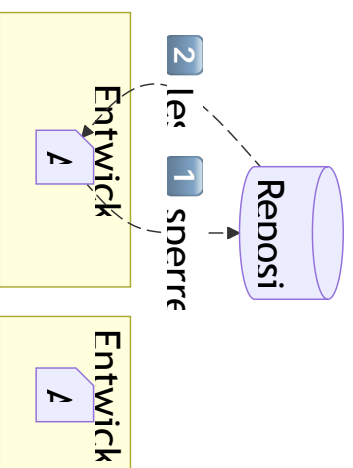
4 Entwickler 2 überschreibt Änderungen von Entwickler 1



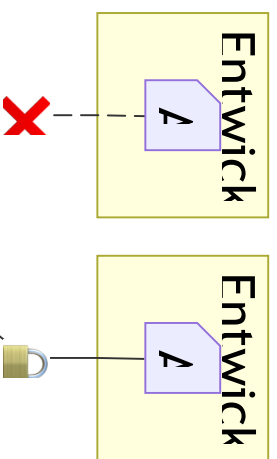
GEMEINSAMES EDITIEREN

LOCK-MODIFY-UNLOCK

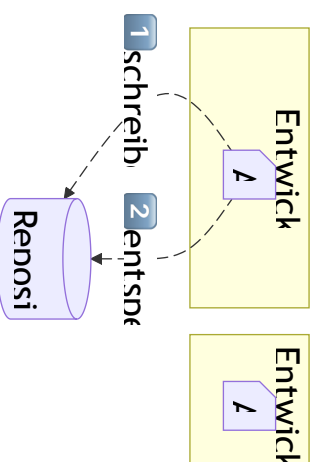
1 Entwickler 1 sperrt die Datei und kopiert sie zum bearbeiten



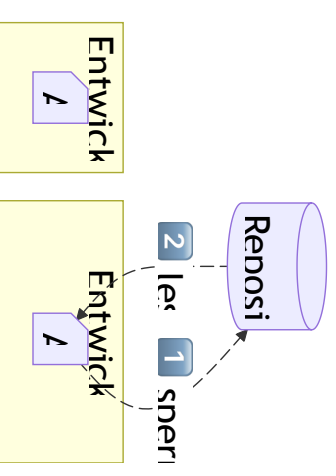
2 Während Entwickler 1 bearbeitet scheitert der Sperr-Versuch von Entwickler 2



3 Entwickler 1 schreibt seine Änderungen und hebt die Sperre auf



4 Jetzt kann Entwickler 2 seine Änderungen vornehmen

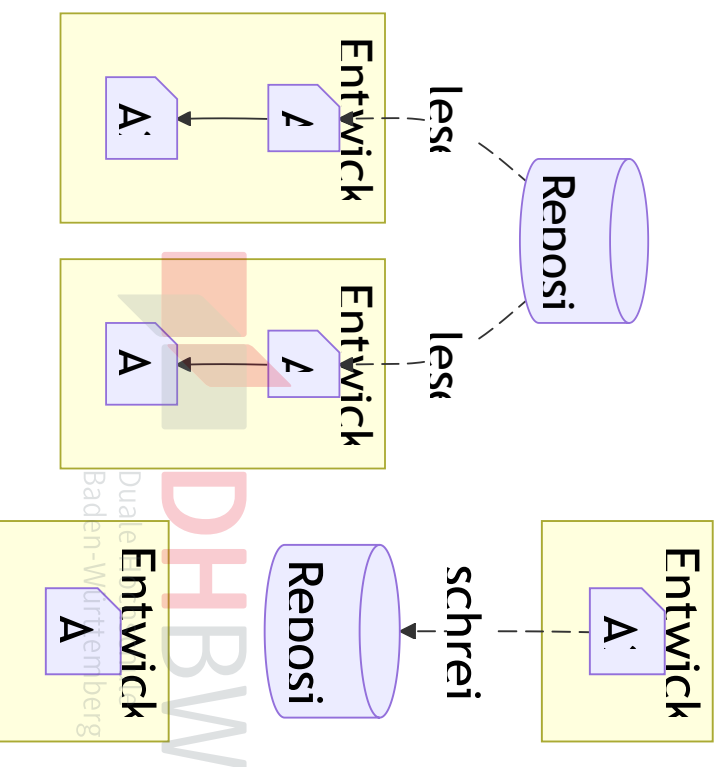


GEMEINSAMES EDITIEREN

COPY-MODIFY-MERGE

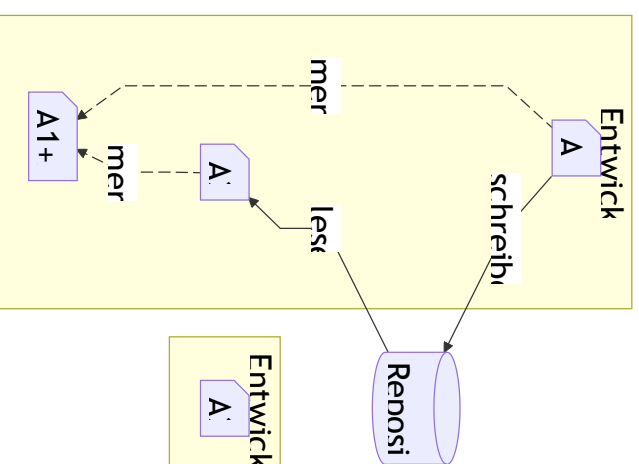
1 Beide Entwickler kopieren die gleiche Datei und bearbeiten diese

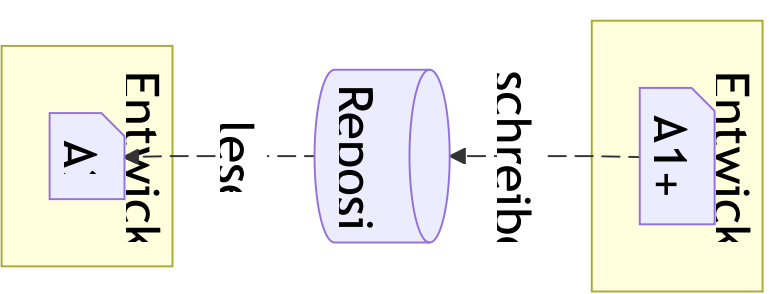
2 Entwickler 2 schreibt seine Änderung zuerst



3 Bevor Entwickler 1 schreiben kann, muss er erst die aktuelle Version übernehmen

4 Entwickler 1 veröffentlicht die neue Version, Entwickler 2 kopiert sie.





DHBW
Duale Hochschule
Baden-Württemberg

KONFLIKTAUFLÖSUNG

✓ Überschneiden sich zwei Änderungen *nicht*, können sie *automatisch* zusammenfließen

⚠ Wenn sich Änderungen im Quellcode jedoch überschneiden, müssen diese Konflikte *manuell* aufgelöst werden

🔧 Damit ein Commit oder Merge erfolgreich ist, müssen diese Konflikte manuell aufgelöst werden.

🔄 Daher immer zuerst neueste Änderungen lokal integrieren, dann Konflikte auflösen und eine neue Revision erstellen.

👉 Zur direkten Auflösung markieren VCS die in Konflikt stehenden Codezeilen (diff).

💻 Außerdem werden *merge-tools* bereitgestellt, meist Texteditoren, in denen die markierten Codezeilen bearbeitet werden können.

ARTEN VON VCS

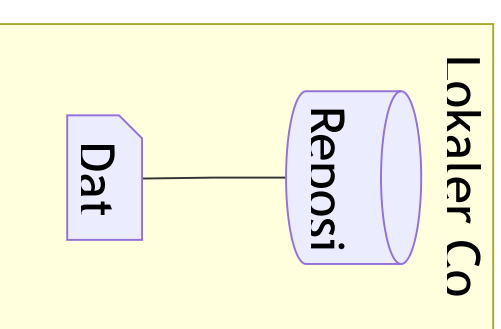
LOKALE VCS

EIGENSCHAFTEN

- Dateien und Repository befinden sich auf der selben Maschine
- Oft nur eine einzige Datei
- Teilweise können alter Versionen in der Datei selbst gespeichert sein
- Beispiele: SCCS, RCS

VOR- UND NACHTEILE

- ✓ Einfach zu handhaben
- ✓ Für einfache Dokumente ausreichend
- ✗ Keine simultane Bearbeitung mit Anderen
- ✗ Keine Versionsverwaltung über mehrere Geräte hinweg



ARTEN VON VCS

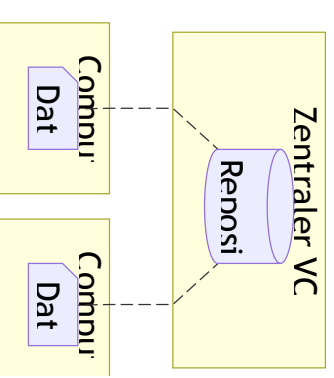
ZENTRALE VCS

EIGENSCHAFTEN

- Es gibt ein zentrale Repository
- Arbeitskopie und Repository befinden sich auf unterschiedlichen Maschinen
- Client-Server-Architektur
- Beispiele: CVS, SVN

VOR- UND NACHTEILE

- ✓ Man arbeitet nie im Repository
- ✓ Einfache Rechte Verwaltung
- ✓ Ausfallsicherheit der lokalen Maschinen
- ✗ Netzwerkzugriff erforderlich
- ✗ Single Point of Failure
- ✗ Client muss Änderungen mit dem Server synchronisieren



ARTEN VON VCS

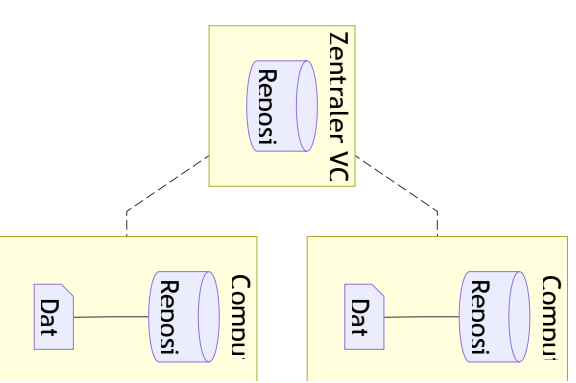
DEZENTRALE VCS

EIGENSCHAFTEN

- Zentrales und lokale Repositories
- Neue Versionen werden zunächst lokal erstellt und später zentral zusammengeführt
- Zusätzliche zentrale Repository-Hierarchien möglich (bei großen Projekten)
- Zusätzliche Operationen zum Austausch zwischen Repositories: pull und push
- Beispiele: Git, Mercurial, Bazaar

VOR- UND NACHTEILE

- ✓ Offline-Arbeit möglich
- ✓ Schnellere Ausführzeit da lokal
- ✓ Keine Abhängigkeit von einzelner Maschine
- ✓ Änderungen können komplexe Hierarchien durchlaufen
- ✗ Repositories müssen zusammengeführt werden (komplizierte Konfliktauflösung)



ORGANISATION VON VCS

- Je nach Team- und Projektgröße gibt es unterschiedliche Organisationsformen
- Die Organisation kann sich auf...
 - ...die Struktur des Repositories auswirken
 - ...die Art der Zusammenarbeit auswirken
 - ...Berechtigungskonzepte, unterschiedliche Zugriffsrechte und Rollen auswirken
- Wie genau die Organisation aussieht hängt von...
 - ...der Größe des Teams ab
 - ...der Art des Projektes ab
 - ...den Anforderungen an das Projekt ab
 - ...bestimmten Standards ab



KONZEPTE UND BEGRIFFE



DHBW
Duale Hochschule
Baden-Württemberg

Begriff	Beschreibung
Repository	Ablage oder Archiv für versionskontrollierte Dateien
Revision / Commit	Eine Version (nach Übergabe einer Änderung)
Branch	Entwicklungszweig(e) des Projektes
Master / Trunk	Hauptentwicklungszweig
Branching	Abzweigen eines Entwicklungszweiges
Merging	Zusammenführen von Entwicklungszweigen oder Dateien
Forward Integration	Merge von Trunk in Branch
Reverse Integration	Merge von Branch in Trunk
Tag	Markierung einer bestimmten Version (z.B. als Release)
Fork	Abspaltung in ein neues unabhängiges Projekt
Merge Conflict	Konflikt bei Zusammenführung von Änderungen

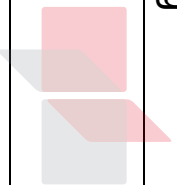


GRUNDBEFEHLE VON VCS



DHBW
Duale Hochschule
Baden-Württemberg

Befehl	Beschreibung
Create/Repo	Erzeuge eine neue Ablage (Repository)
Add/Import	Füge eine Datei oder ein Dateisystem erstmalig der Ablage hinzu
Revision	Zeigt die aktuelle Version einer Datei oder eines Verzeichnisses
Checkout	Erzeuge eine lokale Arbeitskopie einer Datei aus der Ablage
Revert	Verwerfe lokale Änderungen und lade die letzte Version
UpdateSync/Sync	Aktualisiere deine lokale Arbeitskopie
Resolve	Zur Konfliktauflösung bei unterschiedlichen Dateiversionen
CommitCheckin/ Checkin	Lade die Änderungen deiner lokalen Arbeitskopie in die Ablage
Status	Zeigt den aktuellen Stand: Änderungen, Konflikte etc.
Diff/Change/Delta	Zeige die Unterschiede zwischen zwei Versionen
Log/History/ Change log	Auflistung der Veränderungen
Branch	Erstelle eine separate Kopie des Projektes (neuer Entwicklungszweig)
Merge	Integriere die Änderungen eines Branch in einen anderen



BEKANNTE VCS BEISPIELE



APACHE SUBVERSION (SVN)

- Zentrales VCS
- Open Source
- Fortlaufende, strikte Revisionsnummer 🖱️ Revisionskette
- 🛠️ Tools: TortoiseSVN (Windows Explorer), Subclipse (Eclipse)

GIT



- Dezentrales VCS, Open Source, sehr Mächtig
- Referenz auf Vorgängerversion anstatt Revisionsnummer
- Versionen können auf mehrere Vorgänger aufbauen
- Keine implizite Reihenfolge der Versionen 🖱️ Gerichteter azyklischer Graph
- 🛠️ Tools: TortoiseGit (Windows Explorer), GitKraken, EGit (Eclipse)

GIT



ARBEITEN MIT GIT

LOCAL REPOSITORY

💡 In Git wird immer das komplette Remote-Repository in ein Local-Repository auf dem Computer des Entwicklers ausgecheckt (*cloning*).

💿 Das vollständige Abbild eines Repositories liegt lokal im *working directory*.

⚙️ Das Local-Repository besitzt alle Funktionalitäten eines VCSs: committing, branching, merging usw.

🔌 Man arbeitet offline; d.h. man committed in das lokale Repository.

🔄 Nach getaner Arbeit synchronisiert man alle Änderungen zwischen Remote- und Local-Repository.

ARBEITEN MIT GIT

WAS GEHÖRT IN'S REMOTE REPOSITORY

- | | | | |
|---|--|---|--|
| ✓ | Quellcode (z.B. <code>.java</code> , <code>.py</code> , <code>.c</code> , <code>.cpp</code> , <code>.js</code>) | ✗ | Build-Artefakte (z.B. <code>.class</code> , <code>.exe</code> , <code>.dll</code> , <code>.o</code> , <code>.so</code>) |
| ✓ | Textdateien (z.B. <code>.txt</code> , <code>.md</code> , <code>.json</code> , <code>.xml</code> , <code>.yaml</code>) | ✗ | Abhängigkeiten (z.B. Bibliotheken, Frameworks) |
| ✓ | Skripte (z.B. <code>.sh</code> , <code>.bat</code> , <code>.ps1</code>) | ✗ | Konfigurationsdateien mit sensiblen Daten (z.B. Passwörter, API-Schlüssel) |
| ✓ | Build-Skripte (z.B. <code>Makefile</code> , <code>build.gradle</code> , <code>pom.xml</code>) | ✗ | Temporäre Dateien (z.B. Log-Dateien, Cache-Dateien) |
| ✓ | Dokumentation (z.B. <code>.md</code>) | ✗ | IDE-spezifische Dateien (z.B. <code>.vscode</code> , <code>.idea</code>) |
| | | ✗ | Betriebssystem-spezifische Dateien (z.B. <code>.DS_Store</code> , <code>Thumbs.db</code>) |

ARBEITEN MIT GIT

GITIGNORE



DHBW
Duale Hochschule
Baden-Württemberg

- Die Datei `.gitignore` legt fest, welche Dateien und Verzeichnisse von Git ignoriert werden sollen.
- Dies ist nützlich, um temporäre Dateien, Build-Artefakte oder sensible Informationen vom Repository auszuschließen.
- Die `.gitignore`-Datei wird im Stammverzeichnis des Repositories abgelegt und kann für jedes Projekt individuell angepasst werden.
- Beispielinhalte einer `.gitignore`-Datei:
 -  `*.log` - Ignoriert alle Log-Dateien
 -  `build/` - Ignoriert das gesamte Build-Verzeichnis
 -  `*.class` - Ignoriert alle Java-Klassendateien
 -  `.env` - Ignoriert Umgebungsdateien mit sensiblen Daten
 -  `.vscode/` - Ignoriert VSCode-spezifische Einstellungen

 Es gibt viele vorgefertigte `.gitignore`-Dateien für verschiedene Programmiersprachen und Frameworks, die online verfügbar sind

ARBEITEN MIT GIT

BRANCHING

- Branching bedeutet, die Software von der Hauptentwicklungslinie abzuzweigen und daran zu arbeiten, ohne etwas am Hauptentwicklungszweig zu verändern.
- Im Gegensatz zu anderen VCS ermutigt Git zu einer Arbeitsweise mit häufigem Branching und Merging.
- Verwendung:
 - 👉 Für jedes Feature oder Bugfix ein neuer Branch
 - 👉 Wird danach in den Haupt-Entwicklungszweig integriert (Merge)
 - 👉 Branches zur Wartung älterer Versionen

⚠️ Branches müssen immer vor dem Arbeiten (vor der ersten Modifikation) angelegt werden

ARBEITEN MIT GIT

BRANCHING STRATEGIEN

- Es gibt verschiedene Strategien für das Arbeiten mit Branches in Git
 - Die Wahl der richtigen Strategie hängt von Teamgröße, Projektart und Deployment-Zyklen ab
 - Beliebte Ansätze:
 -  Feature Branching
 -  GitFlow
 -  Trunk-Based Development
 - Keine universell "richtige" Strategie - abhängig vom Kontext
-  Weiterführende Informationen: [Git Branching Strategies Guide](#)

ARBEIT MIT GIT

MERGING SZENARIEN

- **Fast-forward merge**

Wenn der Branch, in den gemerged wird, keine neuen Commits hat, wird der Branch einfach auf den Ziel-Branch verschoben.

- **Three-way merge**

Wenn beide Branches neue Commits haben, wird ein Merge-Commit erstellt, der die Änderungen zusammenführt.

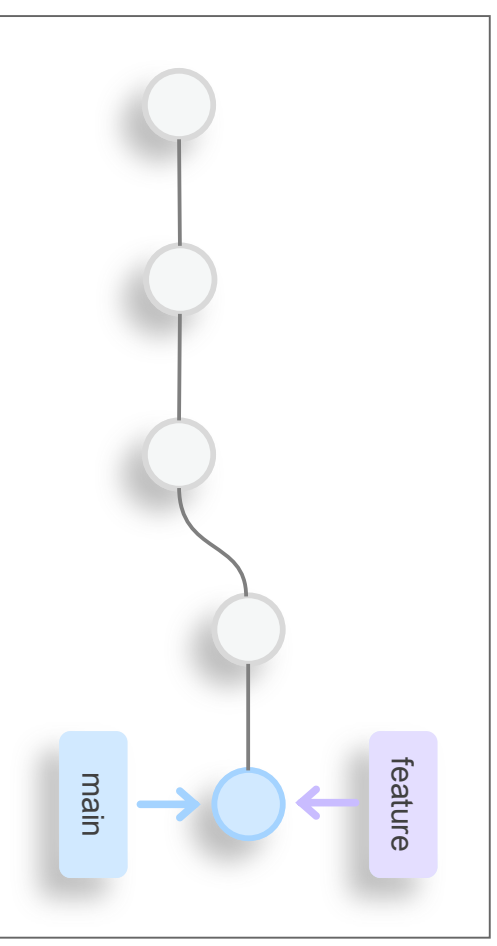
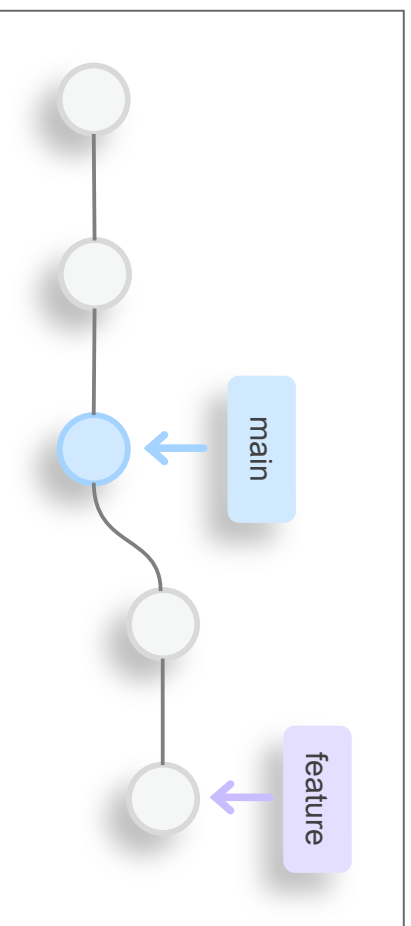
- **Rebase**

Eine alternative Methode zum Merging, bei der die Commits eines Branches auf einen anderen Branch angewendet werden, um eine lineare Historie zu erzeugen.

⚠️ Rebase kann die Historie verändern und sollte mit Vorsicht verwendet werden, insbesondere bei gemeinsam genutzten Branches.

ARBEIT MIT GIT

FAST-FORWARD-MERGE



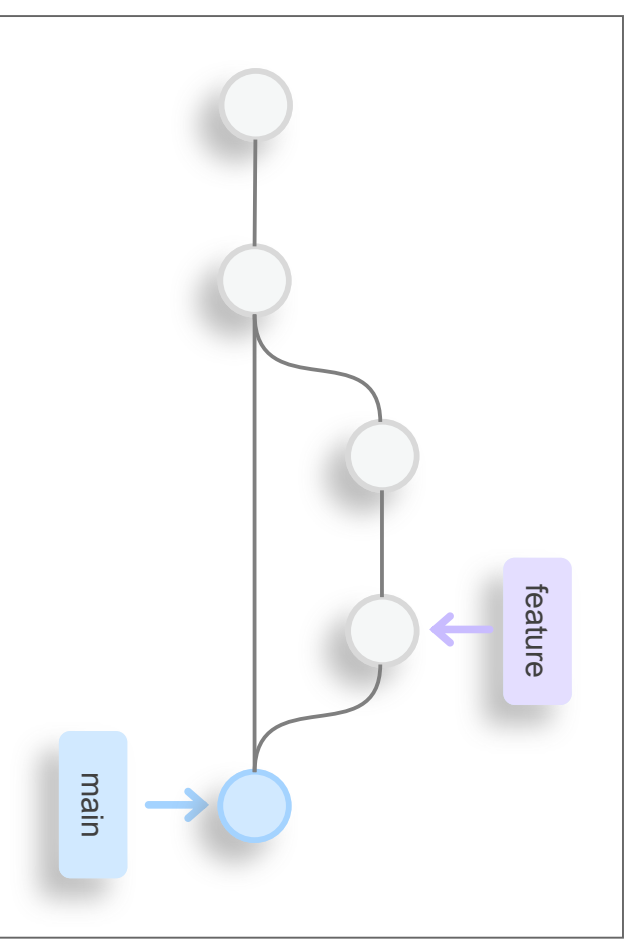
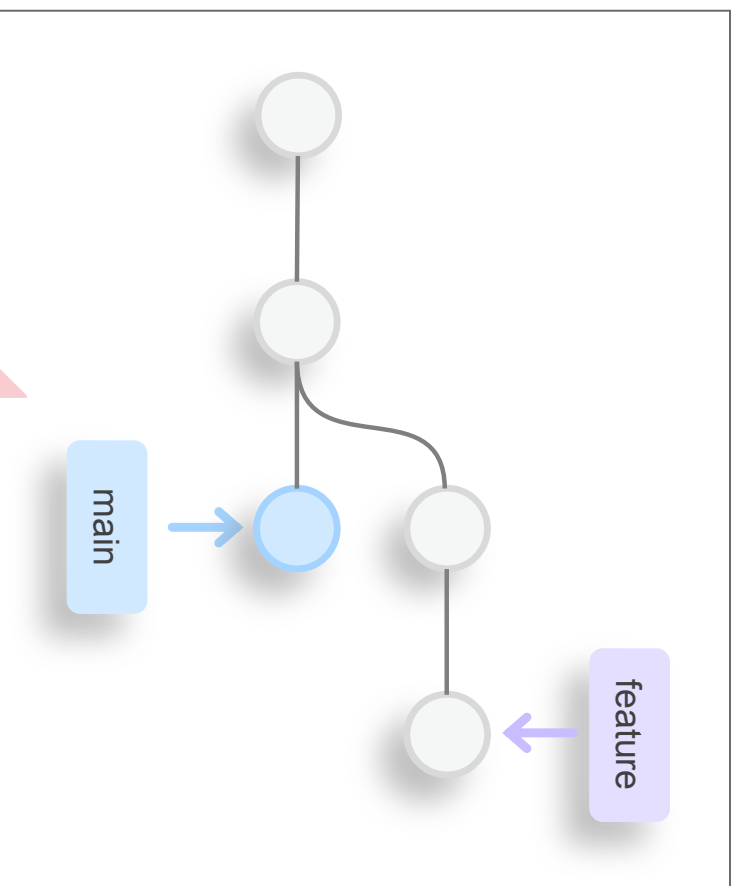
Vor dem Merge

Nach dem Merge

⚠️ Fast-Forward-Merges sind nur möglich, wenn der Ziel-Branch keine neuen Commits hat.

ARBEIT MIT GIT

THREE-WAY-MERGE



Vor dem Merge



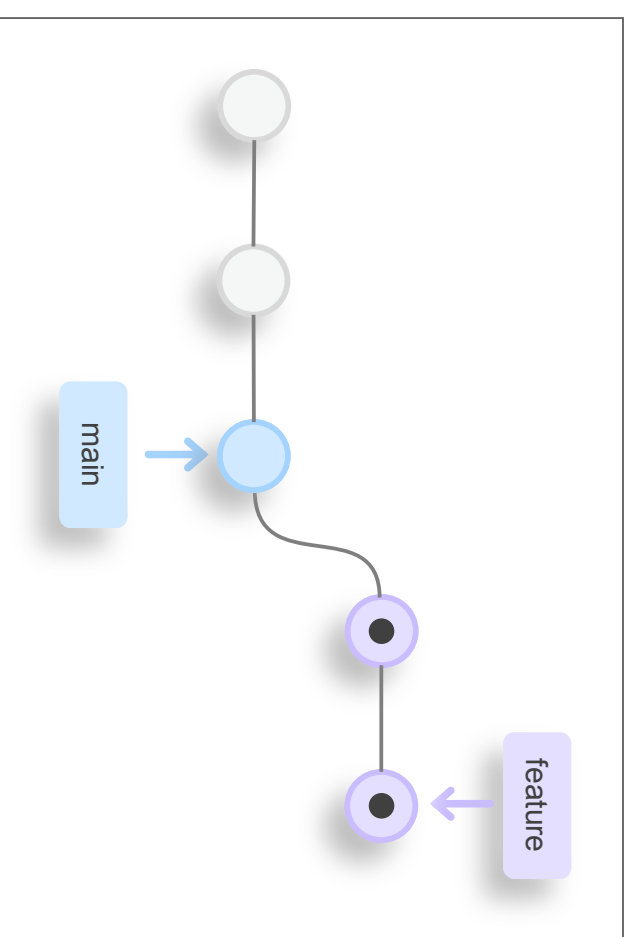
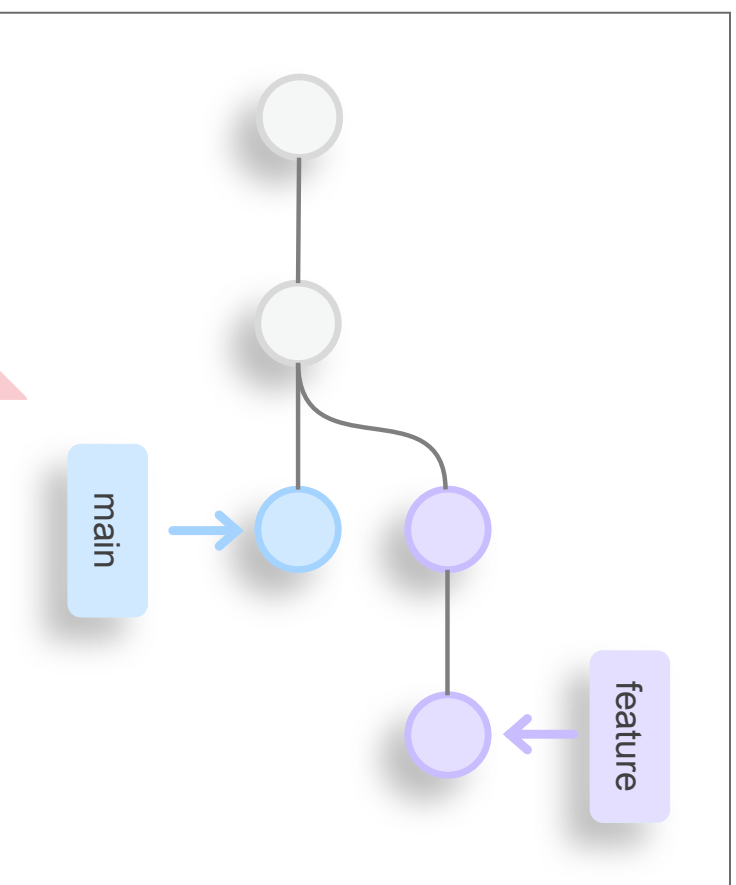
DHBW
Duale Hochschule
Baden-Württemberg

Nach dem Merge

! Three-Way-Merges sind notwendig, wenn beide Branches neue Commits haben.

ARBEIT MIT GIT

REBASE



Vor dem Rebase



DHBW
Duale Hochschule
Baden-Württemberg

Nach dem Rebase

⚠ Rebase verändert die Historie des Branches und sollte mit Vorsicht verwendet werden.

ARBEIT MIT GIT



WICHTIGSTE BEFEHLE

Git-Befehl	Beschreibung
<code>git clone <URL></code>	Kopiert ein bestehendes Repository von einer URL auf den lokalen Rechner.
<code>git fetch</code>	Lädt die neuesten Änderungen aus dem Remote-Repository, ohne sie in den aktuellen Branch zu integrieren.
<code>git pull</code>	Lädt die neuesten Änderungen aus dem Remote-Repository und integriert sie in den aktuellen Branch.
<code>git push</code>	Überträgt lokale Commits in das Remote-Repository.
<code>git add</code> <code><Datei/Ordner></code>	Fügt geänderte Dateien zur Staging-Area hinzu, um sie für den nächsten Commit vorzubereiten.
<code>git commit -m "Nachricht"</code>	Erstellt einen neuen Commit mit den Dateien in der Staging-Area und einer Beschreibung der Änderungen.
<code>git status</code>	Zeigt den aktuellen Status des Repositories, inklusive uncommitteten Änderungen und Staging-Area.

Git-Befehl	Beschreibung
<code>git checkout <Branch/Commit></code>	Wechselt zu einem anderen Branch oder einem bestimmten Commit.
<code>git revert <Commit></code>	Macht einen bestimmten Commit rückgängig, indem ein neuer Commit erstellt wird, der die Änderungen umkehrt.
<code>git diff</code>	Vergleicht Änderungen zwischen Dateien, Staging-Area und Commits.

ARBEITEN MIT GIT

BEST PRACTICES

 Jedes Code-Projekt sollte zumindest in einem lokalen Repository verwaltet werden:

`git init` als erste "Amtshandlung"

 Nach jeder "lauffähigen" Änderung committen.

 Aussagekräftige Commit-Nachrichten schreiben: "Was wurde geändert" und "Warum"

 Im Team: eigener Branch für jedes neue Feature oder Bug Fix

 Nicht direkt in den main/master Branch committen

 Regelmäßig `git pull` ausführen, um Konflikte zu vermeiden

 Keine großen Binärdateien (Bilder, Videos, ZIPs, ...) committen: Verwendung von
• `git ignore` für automatisches Ausschließen

 **Verwendung von Git zur Gewohnheit machen!**

ARBEITEN MIT GIT

ÜBUNG - COMMIT MESSAGES

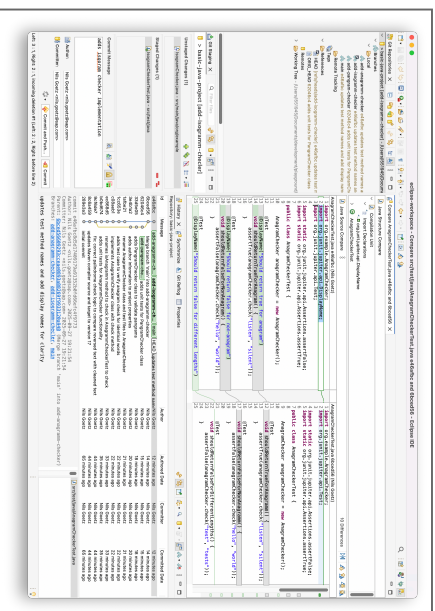
- 👉 fixes bug - ❌
- 👉 adds new function 'getName' for Animal class - ✅
- 👉 fixes null pointer exception in UserService when user is not found - ✅
- 👉 added some stuff - ❌
- 👉 updates README - ❌
- 👉 improve performance of data processing algorithm - ✅
- 👉 Add user authentication and update UI styles - ❌
- 👉 feat(login): implement user login functionality - ✅
- 👉 refactors DatabaseConnection to improve performance and readability - ✅

GIT IN DER IDE

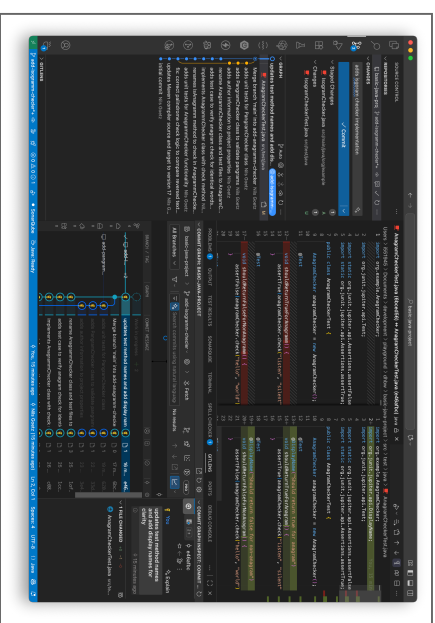
INTELLIJ IDEA



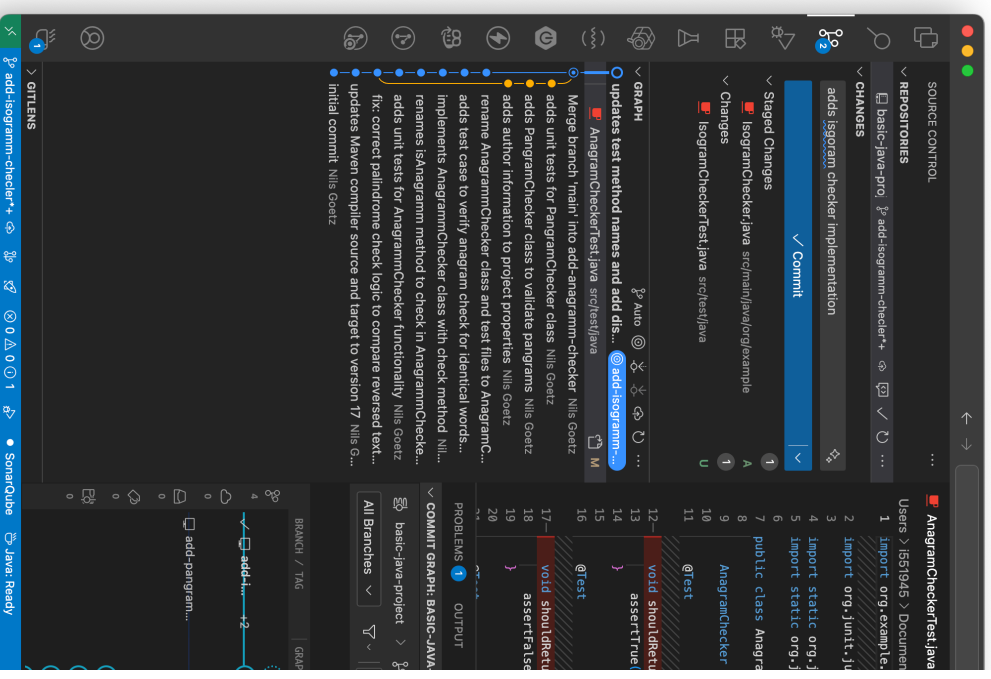
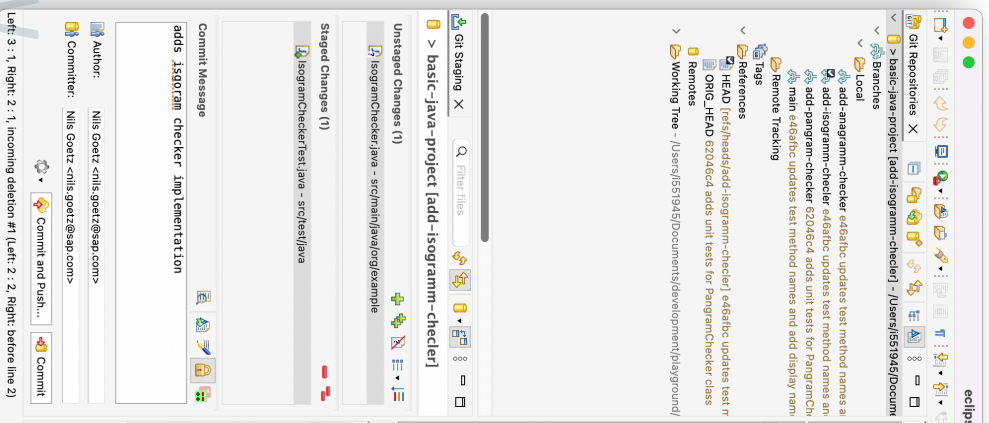
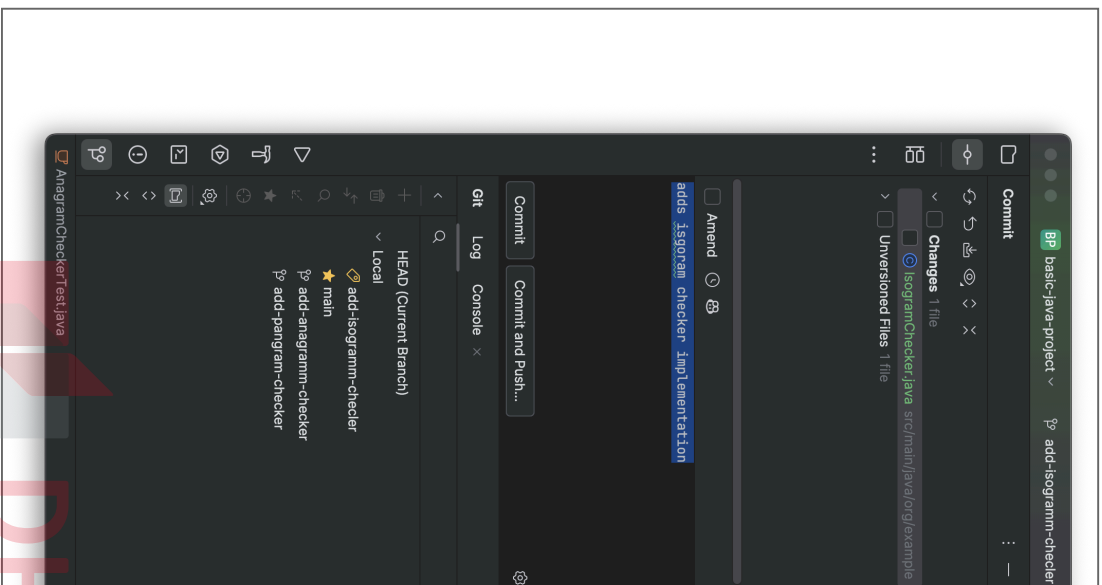
ECLIPSE IDE



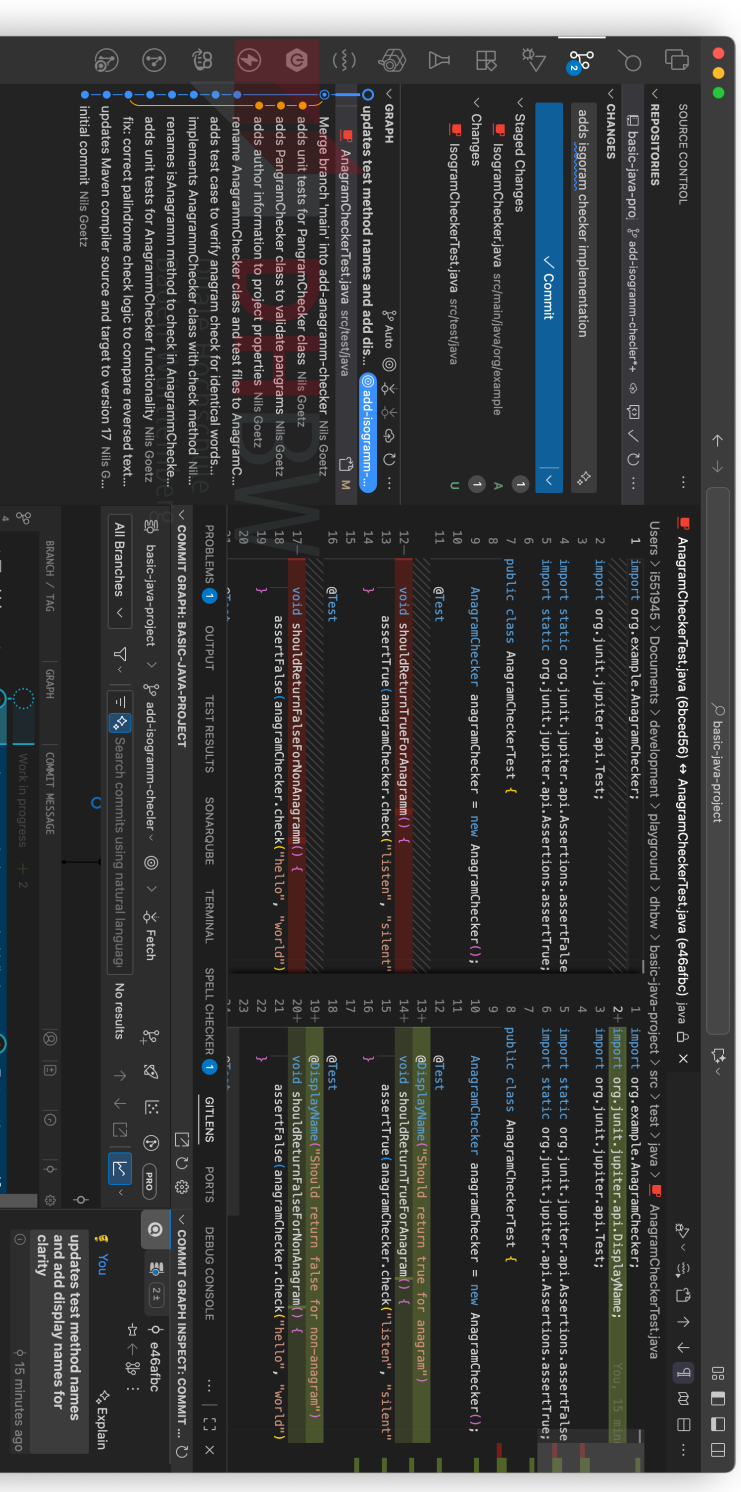
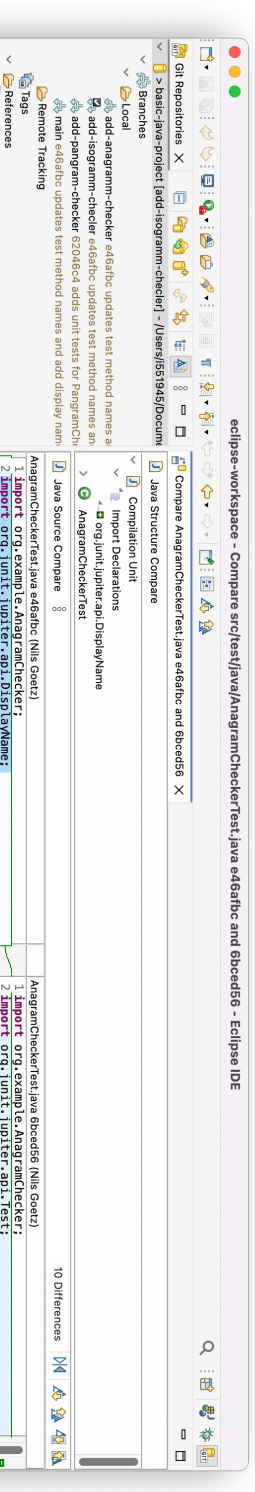
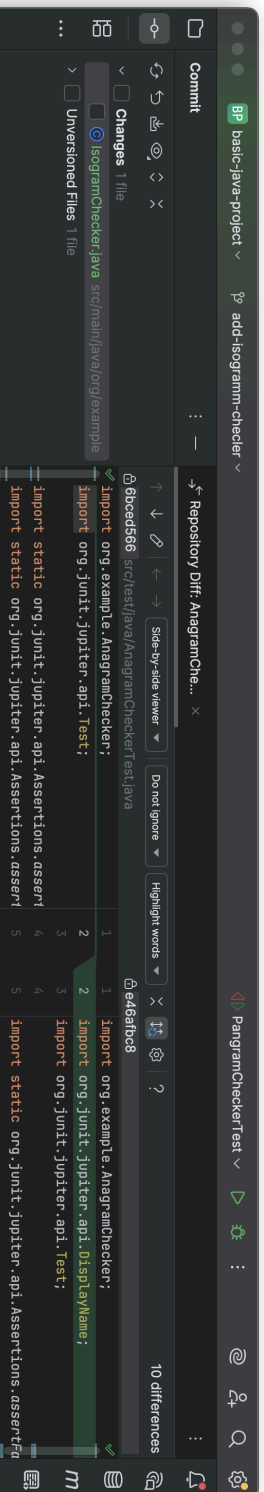
VISUAL STUDIO CODE

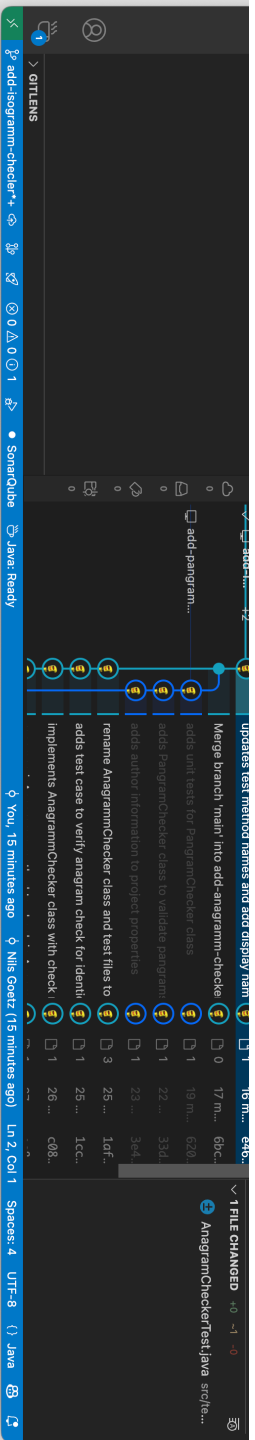


GIT IN DER IDE



GIT IN DER IDE





DHBW
Duale Hochschule
Baden-Württemberg

GITHUB

WAS IST GITHUB?

- GitHub ist eine **webbasierte Hosting-Plattform** für Git-Repositories
- Bietet **kollaborative Entwicklung** mit Features wie Issues, Pull Requests und Code Reviews
- Ermöglicht **öffentliche und private** Projekt-Repositories
- De-facto Standard für **Open-Source-Projekte** und moderne Softwareentwicklung
- Gehört seit 2018 zu **Microsoft**

KERNFUNKTIONEN

- Repository Hosting
- Issue & Bug Tracking
- Pull Requests
- Code Reviews
- GitHub Actions (CI/CD)
- Project Management